

Machine Learning Project Report: Musical Instrument Image Classification



Team Memmbers:

Ali Khaled Elemam 22P0260

Ahmad Sherif Elbaz 2201075

Introduction & Problem Statement

The objective of this project is to develop an automated system for classifying musical instruments from images. Recognizing musical instruments is a challenging computer vision task due to the high intra-class variability (different types of guitars or drums) and inter-class similarity (various types of flutes or percussion instruments). This project utilizes classical machine learning algorithms paired with advanced feature extraction techniques to address this 30-class classification problem.

Methodology & Implementation

The code implements a musical instrument image classification pipeline using classical ML. It loads and resizes images from 30 classes, extracts HOG features, standardizes them, and splits the data into training and test sets. Multiple classifiers—including Random Forest, SVM, KNN, Extra Trees, and XGBoost—are trained and evaluated using accuracy, confusion matrices, and classification reports. This is a feature-based, non-deep learning approach to recognize instruments from images.

1: Dataset Loading and Preprocessing

- Uses the kagglehub library to download the dataset: `gpiosenka/musical-instruments-image-classification`.
- Defines 30 musical instrument classes, e.g., Didgeridoo, Tambourine, Xylophone, etc.
- Iterates through dataset subfolders: 'train', 'valid', 'test'.
- Loads images using OpenCV (`cv2.imread`) and resizes them to 128×128.
- Stores images in `images` and labels as numeric indices in `labels`.

2: Feature Extraction

In this study, images are represented using Histogram of Oriented Gradients (HOG) features, a classical computer vision technique designed to capture shape and structural information. HOG describes objects based on the distribution of local intensity gradients, making it effective for distinguishing between musical instruments with different outlines, textures, and spatial structures. The extracted features are numerical, fixed-length, and independent of color information.

Feature extraction process:

- Image resizing
 - All images are resized to 128×128 pixels
 - Ensures consistent feature vector length across all samples
- Grayscale conversion
 - Images are converted from RGB to grayscale
 - HOG operates on intensity gradients rather than color values
 - Reduces computational cost and focuses on shape information
- Gradient computation
 - Gradient magnitude and direction are computed at each pixel
 - Detects edges and contours such as instrument outlines, strings, and keys

- Cell division

- Image is divided into 16×16 pixel cells
- Produces $8 \times 8 = 64$ cells per image
- Each cell represents a local spatial region

- Orientation histograms

- For each cell, a histogram of 9 gradient orientation bins is created
- Each bin corresponds to a specific range of edge directions

- Block formation

- Cells are grouped into 2×2 blocks
- Improves robustness to illumination and contrast variations
- Each block produces 36 features ($2 \times 2 \times 9$)

- Block normalization

- L2-Hys normalization is applied to each block
- Reduces the effect of lighting changes and enhances generalization

- Final feature vector size

- Number of blocks per image: $7 \times 7 = 49$
- Total features per image: $49 \times 36 = 1764$

- Final feature matrix shape: (number of images, 1764)

Conceptual meaning of the features:

- Encodes edge direction and strength
- Captures local texture patterns
- Preserves spatial layout and overall object shape
- Enables discrimination between different musical instruments

Not extracted:

- Raw pixel values
- Color information
- Deep learning (CNN) features
- Audio-based features

3: Train-Test Split and Scaling

- Splits dataset into 80% training / 20% test using train_test_split with stratification.
- Standardizes features using StandardScaler to improve model performance.

4: Machine Learning Models

The code trains several classical ML models on HOG features:

a) Random Forest

- RandomForestClassifier(n_estimators=100)
- Trained on X_train, y_train
- Predictions: y_pred_rf
- Evaluation: Accuracy, Confusion Matrix, Classification Report

b) SVM

- SVC(kernel='rbf')
- Trained and evaluated similarly

c) K-Nearest Neighbors (KNN)

- KNeighborsClassifier(n_neighbors=5, metric='euclidean', weights='distance')
- Closer neighbors have more influence on predictions

d) Extra Trees

- `ExtraTreesClassifier(n_estimators=300, n_jobs=-1)`
- Fully grown trees for robust performance

e) XGBoost

- `XGBClassifier(n_estimators=400, max_depth=6, learning_rate=0.05, colsample_bytree=0.3, subsample=0.8)`
- Multiclass classification with softmax objective

5: Evaluation

For each model, the following metrics are computed:

- Accuracy
- Confusion Matrix
- Classification Report (Precision, Recall, F1-score per class)

Confusion Matrices

SVM and Random Forest:

```
Number of images loaded: 5093
Features shape: (5093, 1764)
Random Forest Accuracy: 0.47006869479882235
SVM Accuracy: 0.5662414131501472
Random Forest Confusion Matrix:
```

```

[[ 2 0 0 4 4 1 1 0 1 0 0 1 0 1 1 4 0 0 0 2 0 1 0 0
  4 2 1 1 0 0]
[ 2 38 2 1 0 0 0 0 1 0 0 2 1 1 1 0 1 0 1 1 0 0 1 0
  0 1 0 1 1 1]
[ 0 5 22 1 0 0 0 0 3 0 0 5 1 1 0 1 0 0 1 0 1 1 0 0
  0 0 0 0 0 1]
[ 1 2 0 31 0 0 0 0 3 0 0 1 1 0 1 2 0 0 0 1 0 0 0 0
  1 0 0 0 0 0]
[ 4 1 0 0 17 1 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0
  2 1 1 0 0 0]
[ 0 3 1 2 5 2 0 0 2 0 0 0 1 0 0 2 0 1 0 1 0 0 0 0
  7 2 0 0 0 0]
[ 1 5 1 0 1 1 15 0 0 0 0 2 0 0 0 1 0 0 0 0 0 0 0 1
  2 1 0 1 0 1]
[ 0 0 0 2 0 0 0 25 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0
  1 0 0 0 0 0]
[ 0 2 2 0 3 0 0 0 31 0 0 2 0 1 0 1 0 1 0 1 0 0 0 0
  0 0 1 0 0 1]
[ 0 1 0 0 0 1 1 0 2 14 0 1 5 0 0 0 0 0 0 1 0 0 0 1
  0 0 0 0 1 0]
[ 0 0 2 0 3 1 1 0 4 0 4 0 2 2 0 1 2 0 2 0 0 0 1 1
  1 0 1 0 1 0]
[ 1 0 1 0 5 0 0 1 0 0 0 32 1 0 0 0 0 0 0 0 0 0 0 0
  3 0 0 0 0 0]
[ 0 2 0 5 0 0 0 0 1 1 0 0 20 1 0 0 0 0 0 1 0 0 0 0
  0 1 0 0 0 0]
[ 0 0 0 1 0 0 0 0 1 0 0 0 0 36 0 1 0 0 0 0 0 0 0 0
  0 0 0 0 0 0]
[ 0 0 2 1 5 1 2 0 1 1 1 2 2 0 1 1 1 0 0 2 0 0 0 0
  1 0 1 0 2 0]
[ 3 2 1 1 0 0 0 0 3 0 2 3 0 2 1 8 0 1 1 1 0 1 2 0
  4 1 1 1 0 2]
[ 1 1 2 0 2 0 0 1 3 0 0 1 0 0 0 0 5 0 2 1 0 1 0 1
  1 0 1 2 0 0]
[ 1 1 0 0 2 3 1 0 1 0 0 0 1 2 0 0 0 11 0 1 0 0 0 1
  3 0 1 0 0 0]
[ 0 3 4 0 0 0 0 0 1 0 0 4 0 0 0 0 0 0 14 0 0 1 0 0
  0 0 1 1 1 0]
[ 0 0 0 1 5 1 0 0 0 0 0 0 1 0 0 0 0 0 0 26 0 0 0 1
  2 0 0 0 0 0]
[ 0 1 1 1 0 0 0 0 6 0 0 0 0 0 0 1 1 0 0 2 9 0 1 1
  1 0 0 0 0 0]
[ 2 3 1 0 0 0 0 0 5 1 0 1 1 0 0 1 0 0 0 1 1 4 1 0
  3 1 1 0 0 0]
[ 0 0 0 0 0 0 0 1 0 0 0 17 0 1 0 2 0 0 0 0 0 0 3 0
  0 0 0 0 1 1]
[ 0 0 0 1 0 3 0 0 1 0 0 2 1 0 0 0 0 0 0 0 0 0 0 17
  2 0 0 0 0 0]
[ 1 3 2 0 3 0 0 0 1 0 0 2 2 5 0 2 0 1 1 0 0 0 0 0
  19 0 0 1 2 0]

```



```

[ 2  2  1  2  0  0  0  0  0  2  0  2  5  1  0  2  0  0  0  1  0  1  1  0
  1  5  0  0  0  0]
[ 1  2  0  2  0  0  0  0  2  0  0  0  0  0  0  2  0  1  1  2  0  0  0  0
  2  0 19  1  0  0]
[ 0  0  0  1  0  1  0  1  1  0  1  2  0  1  0  1  0  0  1  0  0  1  0  0
  0  0  0 24  0  0]
[ 0  0  0  4  0  0  0  0  0  0  0  6  1  2  0  1  0  1  1  2  0  0  0  0
  1  0  0  0 18  0]
[ 1  0  0  8  4  0  0  0  1  0  0  1  0  1  0  1  0  0  1  2  0  0  1  1
  1  0  0  0  1  7]]

```

SVM Confusion Matrix:

```

[[10  0  1  2  3  0  1  0  0  0  1  1  0  0  0  7  1  0  0  1  0  0  0  0
  1  2  0  0  0  0]
[ 0 40  0  0  1  0  1  0  3  1  1  0  1  0  0  3  0  0  0  1  0  0  0  0
  1  1  1  1  0  1]
[ 0  1 26  0  0  1  0  0  3  0  0  3  1  0  1  3  1  0  1  0  0  0  0  0
  0  0  0  1  0  1]
[ 1  0  1 33  0  1  0  0  2  0  0  0  0  0  0  1  0  0  0  0  0  0  1  0
  2  1  0  0  1  0]
[ 6  0  0  0 20  0  0  0  0  0  0  0  0  0  1  0  0  1  0  0  0  0  0  0
  0  1  0  0  0  0]
[ 0  0  2  1  5  5  0  0  2  0  0  0  0  0  0  3  0  0  0  2  0  1  0  0
  6  1  0  0  0  1]
[ 3  3  0  0  1  0 17  0  1  2  0  0  0  0  0  2  1  0  1  0  0  0  0  0
  0  0  0  0  1  1]
[ 0  0  0  0  0  0  0 25  1  0  0  0  0  0  0  1  0  0  0  0  1  0  1  0
  0  0  0  1  0  0]
[ 0  0  0  0  1  0  0  0 37  0  0  1  0  0  0  0  1  0  0  1  0  0  0  1
  0  3  0  0  0  1]
[ 0  1  0  0  0  0  1  0  1 22  0  0  0  0  0  0  1  0  0  1  0  0  0  0
  1  0  0  0  0  0]
[ 0  0  0  0  2  1  0  0  4  0  3  1  0  1  0  5  2  0  0  1  0  0  0  1
  4  0  1  2  0  1]
[ 0  0  1  0  1  0  0  0  0  1  0 34  0  0  0  0  0  1  0  0  0  1  2  0
  2  1  0  0  0  0]
[ 0  1  0  1  1  0  0  0  0  0  0  0 24  1  0  0  0  0  0  1  0  0  0  0
  2  1  0  0  0  0]
[ 1  0  0  1  0  0  0  0  0  0  0  0  0 37  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0]
[ 2  0  1  0  2  1  0  0  1  0  2  2  1  0  5  3  0  0  0  2  0  0  0  0
  2  0  1  0  2  0]
[ 4  1  0  1  0  0  1  0  1  0  2  2  0  0  0 19  0  0  1  1  0  0  0  0
  4  1  1  1  0  1]
[ 0  0  1  0  1  0  0  0  3  0  1  0  0  0  2  3 10  0  0  1  0  0  0  0
  1  0  1  1  0  0]
[ 0  0  0  0  3  0  1  0  1  0  0  0  2  1  0  1  0 12  0  0  0  1  0  0
  5  1  0  0  1  0]
[ 0  0  3  0  0  0  0  0  1  1  0  2  0  0  0  0  0  0 17  0  0  3  0  0
  0  0  0  1  1  1]
[ 3  0  0  0  4  0  0  0  0  0  0  0  0  1  0  1  0  0  0 26  0  1  0  0

```

```

1 0 0 0 0 0]
[ 1 0 1 0 0 0 0 0 4 0 1 1 1 0 0 2 1 0 0 2 7 0 0 0
 2 0 0 1 0 1]
[ 1 1 0 0 2 0 0 0 4 0 0 1 0 0 1 2 1 0 1 1 0 7 1 0
 3 0 0 1 0 0]
[ 0 0 0 0 0 1 0 0 0 0 0 10 0 0 0 3 0 0 2 0 0 2 8 0
 0 0 0 0 0 0]
[ 1 0 0 0 0 1 0 0 0 0 0 1 0 0 0 1 0 1 0 1 0 0 1 18
 2 0 0 0 0 0]
[ 3 1 1 0 2 1 0 0 2 0 1 1 0 0 0 2 0 0 0 1 0 0 0 1
24 1 1 2 0 1]
[ 2 0 0 1 2 1 0 1 0 1 0 1 3 0 1 1 0 0 0 1 0 0 0 0
 3 9 0 0 0 1]
[ 1 0 2 0 0 0 0 0 1 0 0 0 0 0 0 4 0 0 1 1 0 0 0 0
 2 0 21 2 0 0]
[ 0 0 0 1 0 0 0 0 2 0 0 0 0 0 0 3 0 0 0 0 0 0 0 0
 1 0 0 28 0 0]
[ 0 1 1 2 0 1 0 0 0 0 0 1 0 0 1 2 0 0 0 1 0 0 1 0
 1 1 1 0 23 0]
[ 1 0 1 0 3 0 0 0 1 0 0 1 0 1 0 0 0 0 1 3 0 0 1 1
 5 1 0 1 0 10]]

```

Random Forest Report:

	precision	recall	f1-score	support
Didgeridoo	0.09	0.06	0.07	31
Tambourine	0.49	0.67	0.57	57
Xylophone	0.49	0.51	0.50	43
acordian	0.45	0.70	0.55	44
alphorn	0.29	0.59	0.39	29
bagpipes	0.12	0.07	0.09	29
banjo	0.71	0.45	0.56	33
bongo drum	0.86	0.83	0.85	30
casaba	0.41	0.67	0.51	46
castanets	0.70	0.50	0.58	28
clarinet	0.50	0.14	0.22	29
clavichord	0.36	0.73	0.48	44
concertina	0.43	0.62	0.51	32
drums	0.61	0.92	0.73	39
dulcimer	0.20	0.04	0.06	27
flute	0.22	0.20	0.21	41
guiro	0.50	0.20	0.29	25
guitar	0.65	0.38	0.48	29
harmonica	0.54	0.47	0.50	30
harp	0.53	0.70	0.60	37
marakas	0.75	0.36	0.49	25
ocarina	0.36	0.15	0.21	27
piano	0.27	0.12	0.16	26
saxaphone	0.68	0.63	0.65	27
sitar	0.31	0.42	0.36	45
steel drum	0.33	0.18	0.23	28

trombone	0.66	0.54	0.59	35
trumpet	0.73	0.69	0.71	35
tuba	0.64	0.49	0.55	37
violin	0.50	0.23	0.31	31
accuracy			0.47	1019
macro avg	0.48	0.44	0.43	1019
weighted avg	0.48	0.47	0.45	1019

KNN:

KNN Accuracy: 0.48773307163886165

KNN Confusion Matrix:

```

[[15  0  0  0  3  2  0  0  0  0  1  1  0  0  3  2  0  0  0  1  0  0  0  0
  3  0  0  0  0  0  0]
 [ 3 37  0  1  0  0  0  1  3  2  0  0  6  1  1  0  1  0  0  0  0  0  0  0
  0  0  0  1  0  0]
 [ 1  1 20  1  2  0  0  1  1  0  1  6  3  0  2  1  0  0  1  0  0  1  1  0
  0  0  0  0  0  0]
 [ 7  1  0 28  1  1  0  0  1  0  1  0  3  0  0  0  0  0  0  0  0  0  1  0
  0  0  0  0  0  0]
 [ 8  0  0  0 15  0  0  0  0  0  0  1  0  0  0  2  0  0  0  0  2  0  0  0
  0  0  0  0  1  0]
[12  1  0  2  1  4  0  0  0  0  0  0  2  1  0  2  0  1  0  0  0  1  0  0
  0  1  0  0  0  1]
 [ 5  2  0  1  0  0 17  0  1  0  0  1  1  0  1  0  0  2  0  0  0  0  0  1
  0  0  0  1  0  0]
 [ 1  0  0  2  0  0  0 24  0  0  1  0  0  0  0  1  0  0  0  0  0  0  1  0
  0  0  0  0  0  0]
 [ 3  1  2  1  0  0  0  1 25  0  1  1  4  0  0  1  0  1  0  1  1  0  0  1
  0  1  0  0  0  1]
 [ 2  1  0  0  1  3  1  1  0 14  0  0  3  0  0  0  0  0  0  0  1  0  0  0
  1  0  0  0  0  0]
 [ 4  0  0  0  3  1  0  1  0  0  4  1  1  1  1  3  1  1  1  0  0  0  0  3
  1  0  1  1  0  0]
 [ 5  0  0  0  4  0  0  0  0  0  0 30  3  0  0  2  0  0  0  0  0  0  0  0
  0  0  0  0  0  0]
 [ 2  1  0  0  1  0  0  0  0  0  0  0 27  1  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0]
 [ 1  0  0  0  0  1  0  0  0  0  0  0  0 37  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0]
 [ 7  0  0  0  2  1  1  0  0  0  2  2  0  1  4  2  0  1  1  0  0  1  0  0
  0  0  0  0  1  1]
 [ 6  0  0  0  6  2  0  0  1  0  4  3  0  0  2 10  0  0  2  0  0  0  0  0
  2  0  0  2  0  1]
 [ 1  0  1  1  1  0  2  1  2  0  1  0  1  0  2  2  5  0  1  0  0  0  0  0
  1  0  0  2  0  1]

```

```

[ 4 0 1 1 1 1 1 0 1 0 1 0 2 0 0 0 0 11 0 0 0 1 0 2
 1 0 0 0 0 1]
[ 0 1 0 0 0 1 0 0 0 0 0 5 0 0 2 0 0 0 19 0 0 1 0 0
 0 0 0 1 0 0]
[ 5 0 0 0 3 0 0 0 0 0 1 0 2 0 1 2 0 0 0 22 1 0 0 0
 0 0 0 0 0 0]
[ 1 0 0 0 1 0 1 0 0 0 0 0 2 2 0 1 0 2 0 1 10 0 0 1
 0 1 0 2 0 0]
[ 5 1 0 0 1 1 0 0 4 0 0 2 1 0 1 0 0 2 0 0 0 6 0 1
 1 0 0 0 1 0]
[ 2 0 0 0 2 1 1 0 0 0 1 5 2 1 1 1 0 0 1 0 0 2 6 0
 0 0 0 0 0 0]
[ 2 0 0 0 3 2 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 18
 1 0 0 0 0 0]
[ 9 0 0 0 0 3 1 0 0 0 3 1 4 1 1 1 0 1 0 0 0 3 0 1
 13 0 1 1 0 1]
[ 8 1 0 0 1 2 1 0 0 0 0 1 4 1 0 0 0 3 0 0 0 2 0 0
 0 4 0 0 0 0]
[ 3 0 0 0 2 0 0 1 0 0 1 0 0 1 0 3 0 0 3 0 0 0 0 1
 0 0 17 3 0 0]
[ 3 0 0 0 0 0 0 0 0 0 1 1 1 0 0 1 0 0 0 0 0 0 0 0
 1 0 0 26 1 0]
[ 4 0 0 0 0 2 0 0 1 0 1 2 3 0 0 2 0 0 0 0 0 0 0 1
 0 0 0 0 21 0]
[ 5 0 0 2 5 1 0 1 0 0 0 1 1 1 0 1 0 1 0 0 0 0 0 1
 0 0 1 0 2 8]]

```

KNN Classification Report:

	precision	recall	f1-score	support
Didgeridoo	0.11	0.48	0.18	31
Tambourine	0.77	0.65	0.70	57
Xylophone	0.83	0.47	0.60	43
acordian	0.70	0.64	0.67	44
alphorn	0.25	0.52	0.34	29
bagpipes	0.14	0.14	0.14	29
banjo	0.65	0.52	0.58	33
bongo drum	0.75	0.80	0.77	30
casaba	0.62	0.54	0.58	46
castanets	0.88	0.50	0.64	28
clarinet	0.16	0.14	0.15	29
clavichord	0.47	0.68	0.56	44
concertina	0.36	0.84	0.50	32
drums	0.76	0.95	0.84	39
dulcimer	0.18	0.15	0.16	27
flute	0.25	0.24	0.25	41
guiro	0.71	0.20	0.31	25
guitar	0.42	0.38	0.40	29
harmonica	0.66	0.63	0.64	30
harp	0.85	0.59	0.70	37
marakas	0.67	0.40	0.50	25

ocarina	0.33	0.22	0.27	27
piano	0.67	0.23	0.34	26
saxophone	0.58	0.67	0.62	27
sitar	0.52	0.29	0.37	45
steel drum	0.57	0.14	0.23	28
trombone	0.85	0.49	0.62	35
trumpet	0.65	0.74	0.69	35
tuba	0.78	0.57	0.66	37
violin	0.53	0.26	0.35	31
accuracy			0.49	1019
macro avg	0.56	0.47	0.48	1019
weighted avg	0.57	0.49	0.50	1019

Extra Trees

xtra Trees Accuracy: 0.5309126594700687

Extra Trees Confusion Matrix:

```

[[10  0  0  1  3  0  1  0  1  0  0  3  0  0  3  1  0  0  1  2  0  0  0  0
  2  2  1  0  0  0]
[ 0 45  1  0  1  0  0  1  2  0  0  0  1  0  0  1  0  1  0  1  0  0  0  0
  1  0  0  1  1  0]
[ 0  3 25  0  0  0  0  0  5  0  0  5  2  0  0  0  0  0  0  1  0  1  0
  1  0  0  0  0  0]
[ 0  0  1 32  0  0  0  0  3  1  0  4  0  0  0  1  0  0  0  1  0  0  0  0
  1  0  0  0  0  0]
[ 1  0  0  0 20  0  0  0  0  1  0  0  1  0  0  0  1  0  0  0  0  0  0  0
  4  0  1  0  0  0]
[ 2  1  2  1  5  1  0  0  2  0  0  1  2  0  0  2  0  0  0  1  0  0  0  0
  6  2  0  0  1  0]
[ 3  5  0  0  0  0 16  0  2  0  0  2  0  0  0  0  0  0  1  0  0  0  0  1
  0  1  0  2  0  0]
[ 0  0  0  0  0  0  0 27  1  0  0  0  0  1  0  0  0  0  1  0  0  0  0  0
  0  0  0  0  0  0]
[ 0  1  1  0  2  0  0  0 36  0  0  2  0  0  0  0  0  1  0  1  0  0  0  1
  0  1  0  0  0  0]
[ 0  4  0  0  0  0  1  0  3 15  0  0  2  0  0  1  0  0  0  1  0  0  0  0
  1  0  0  0  0  0]
[ 0  3  0  0  2  1  1  0  4  0  2  1  0  0  2  3  0  1  0  1  0  0  0  2
  1  0  2  3  0  0]
[ 0  1  0  0  2  0  0  0  1  0  0 35  3  0  0  0  0  0  0  0  0  0  0  0
  2  0  0  0  0  0]
[ 0  1  0  1  0  0  0  0  0  0  0  1 26  2  0  0  0  0  0  0  0  0  0  0
  1  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0 38  0  0  0  0  0  1  0  0  0  0
  0  0  0  0  0  0]

```

```

[ 1 0 1 0 4 0 0 0 1 0 1 3 1 1 4 0 0 0 2 1 0 0 0 1
 3 0 1 0 1 1]
[ 2 2 0 0 2 1 0 0 2 0 2 3 0 1 0 8 0 0 3 1 0 0 1 0
 8 1 2 1 0 1]
[ 0 0 3 0 0 0 0 0 3 0 0 3 0 0 0 1 5 0 2 1 0 1 0 0
 3 1 1 1 0 0]
[ 1 0 0 0 2 0 1 0 1 0 0 0 4 2 0 0 0 15 0 0 0 0 0 0
 1 0 1 0 1 0]
[ 0 3 2 0 0 0 0 0 0 0 0 4 0 0 0 0 1 0 18 0 0 0 1 0
 0 0 0 1 0 0]
[ 1 0 0 1 6 0 0 0 0 1 0 0 1 0 0 0 0 0 25 0 0 0 0
 2 0 0 0 0 0]
[ 0 1 2 1 0 0 0 0 3 0 1 1 1 0 0 0 0 0 2 11 0 0 0
 2 0 0 0 0 0]
[ 1 4 1 3 2 0 1 0 2 0 0 4 0 0 0 1 0 0 0 2 0 4 0 0
 1 0 1 0 0 0]
[ 0 1 0 0 0 0 0 0 1 0 0 17 2 0 0 0 0 0 1 0 0 0 2 0
 1 0 0 0 0 1]
[ 0 0 0 1 1 0 0 0 0 0 0 1 2 0 0 1 0 1 0 1 0 0 0 18
 1 0 0 0 0 0]
[ 0 3 1 1 3 0 1 0 4 0 1 2 0 2 0 0 0 2 1 0 0 0 0 0
 20 0 0 3 1 0]
[ 3 4 0 1 2 2 0 0 2 0 0 2 7 1 0 0 0 0 0 0 0 0 0 0
 0 4 0 0 0 0]
[ 1 1 1 2 0 0 0 0 1 1 0 0 1 0 0 2 0 0 1 0 0 0 0 0
 2 0 21 1 0 0]
[ 0 0 1 0 0 0 0 1 2 0 0 1 0 0 0 1 0 0 1 0 0 0 0 0
 1 0 0 27 0 0]
[ 0 0 0 4 0 0 0 0 0 0 0 7 1 1 0 0 0 0 1 1 0 0 0 0
 0 0 0 0 22 0]
[ 1 1 0 5 2 1 0 0 0 0 0 2 0 0 0 1 0 0 0 2 0 0 0 1
 3 1 1 1 0 9]]

```

Extra Trees Classification Report:

	precision	recall	f1-score	support
Didgeridoo	0.37	0.32	0.34	31
Tambourine	0.54	0.79	0.64	57
Xylophone	0.60	0.58	0.59	43
acordian	0.59	0.73	0.65	44
alphorn	0.34	0.69	0.45	29
bagpipes	0.17	0.03	0.06	29
banjo	0.73	0.48	0.58	33
bongo drum	0.93	0.90	0.92	30
casaba	0.44	0.78	0.56	46
castanets	0.79	0.54	0.64	28
clarinet	0.29	0.07	0.11	29
clavichord	0.34	0.80	0.47	44
concertina	0.46	0.81	0.58	32
drums	0.78	0.97	0.86	39
dulcimer	0.44	0.15	0.22	27

flute	0.33	0.20	0.25	41
guiro	0.83	0.20	0.32	25
guitar	0.68	0.52	0.59	29
harmonica	0.56	0.60	0.58	30
harp	0.54	0.68	0.60	37
marakas	0.92	0.44	0.59	25
ocarina	0.80	0.15	0.25	27
piano	0.40	0.08	0.13	26
saxophone	0.75	0.67	0.71	27
sitar	0.29	0.44	0.35	45
steel drum	0.31	0.14	0.20	28
trombone	0.66	0.60	0.63	35
trumpet	0.66	0.77	0.71	35
tuba	0.81	0.59	0.69	37
violin	0.75	0.29	0.42	31
accuracy			0.53	1019
macro avg	0.57	0.50	0.49	1019
weighted avg	0.56	0.53	0.50	1019

XGBoost:

XGBoost Accuracy: 0.55053974484789

XGBoost Confusion Matrix:

```
[[11  0  0  2  1  0  0  0  1  0  1  0  1  0  1  3  0  0  1  1  0  0  0  1
  3  1  2  0  0  1]
[ 0 41  2  0  0  0  2  0  0  1  0  0  1  0  0  1  2  1  1  3  0  0  0  0
  0  0  1  1  0  0]
[ 0  1 24  1  0  0  0  1  3  0  0  1  0  0  2  2  3  0  1  0  1  2  0  0
  1  0  0  0  0  0]
[ 0  1  1 33  0  0  0  0  2  0  0  1  1  0  0  3  0  0  0  0  0  0  0  0
  0  1  0  0  1  0]
[ 3  0  0  0  0 18  1  0  0  0  0  0  0  0  0  1  2  0  1  0  0  0  0  0
  0  2  0  0  1  0]
[ 2  1  1  3  2  6  0  0  1  0  0  0  0  0  0  1  4  0  0  0  0  2  0  0
  5  1  0  0  0  0]
[ 4  3  0  0  1  0 16  0  1  1  0  0  0  0  0  0  2  0  0  2  0  0  0  0
  2  0  0  0  1  0]
[ 0  0  0  1  0  0  0 27  0  0  0  0  0  0  0  0  1  0  0  0  0  1  0  0
  0  0  0  0  0  0]
[ 0  0  1  1  1  1  0  0 32  0  0  0  1  0  0  0  1  0  0  1  0  1  0  2
  0  1  2  0  0  1]
[ 0  0  0  0  0  1  0  0  1 17  0  1  3  0  0  0  0  1  0  1  0  1  0  0
  0  0  0  0  2  0]
[ 2  0  0  1  0  0  0  0  2  0  5  1  0  1  1  2  2  0  0  1  1  0  0  1
  3  0  3  2  0  1]
[ 0  0  0  0  1  0  0  0  0  2  0 31  1  0  0  1  0  1  1  0  0  1  2  0
  2  1  0  0  0  0]
```

```

[ 0 2 0 2 0 0 0 0 0 0 0 0 0 23 1 2 0 0 0 0 0 0 0 0 0
 0 1 0 0 1 0]
[ 0 0 0 1 0 0 0 0 0 0 0 0 0 0 38 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0]
[ 0 1 0 0 2 0 0 1 1 0 2 0 2 1 3 0 2 0 2 2 0 0 2 0
 3 0 2 0 1 0]
[ 0 1 0 2 0 1 1 0 0 0 1 1 0 0 0 17 2 1 3 1 0 1 0 0
 5 2 1 1 0 0]
[ 0 1 0 1 1 1 0 0 2 0 0 0 0 0 0 3 11 0 1 0 0 0 0 0
 1 0 2 1 0 0]
[ 0 0 0 0 3 0 1 0 1 0 0 0 2 1 0 1 0 14 0 0 0 1 0 0
 3 1 0 0 1 0]
[ 0 2 2 0 0 0 0 0 0 1 0 2 0 0 0 0 0 0 17 0 0 1 0 0
 0 0 1 3 1 0]
[ 0 0 0 0 5 0 0 0 0 1 0 0 0 0 0 1 0 0 0 26 0 0 0 0
 2 1 0 0 0 1]
[ 0 0 1 0 0 0 0 0 3 0 1 0 1 0 0 0 1 0 0 2 9 0 0 0
 2 1 2 2 0 0]
[ 0 0 0 2 0 1 0 0 2 0 1 1 0 0 0 0 1 1 2 1 1 10 0 0
 2 0 0 0 2 0]
[ 0 2 0 0 2 1 0 0 0 0 0 11 0 0 1 2 0 0 1 0 0 0 6 0
 0 0 0 0 0 0]
[ 0 0 1 1 0 1 0 0 0 0 0 1 0 0 0 1 0 1 0 1 0 0 1 17
 2 0 0 0 0 0]
[ 2 2 1 0 0 1 1 1 1 1 0 0 0 1 1 2 0 1 2 0 0 0 0 2
 24 0 0 1 0 1]
[ 0 0 0 1 1 1 0 1 3 1 0 1 3 0 0 3 0 0 0 0 0 0 1 0
 3 8 0 0 0 1]
[ 0 1 1 1 1 0 0 0 0 2 0 0 0 0 0 3 0 0 1 0 2 1 0 0
 1 0 19 2 0 0]
[ 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 3 0 0 0 0 0 0 0 1
 1 0 0 28 0 0]
[ 0 1 1 2 1 1 0 0 0 0 0 3 1 1 0 1 0 0 1 1 0 0 0 0
 1 0 0 0 22 0]
[ 1 0 1 0 2 0 1 0 0 2 0 0 1 1 0 0 0 1 2 3 0 0 0 2
 1 1 1 1 2 8]]

```

XGBoost Classification Report:

	precision	recall	f1-score	support
Didgeridoo	0.44	0.35	0.39	31
Tambourine	0.68	0.72	0.70	57
Xylophone	0.65	0.56	0.60	43
acordian	0.60	0.75	0.67	44
alphorn	0.43	0.62	0.51	29
bagpipes	0.35	0.21	0.26	29
banjo	0.73	0.48	0.58	33
bongo drum	0.84	0.90	0.87	30
casaba	0.56	0.70	0.62	46
castanets	0.59	0.61	0.60	28
clarinet	0.45	0.17	0.25	29

clavichord	0.56	0.70	0.63	44
concertina	0.56	0.72	0.63	32
drums	0.84	0.97	0.90	39
dulcimer	0.23	0.11	0.15	27
flute	0.29	0.41	0.34	41
guiro	0.44	0.44	0.44	25
guitar	0.61	0.48	0.54	29
harmonica	0.45	0.57	0.50	30
harp	0.59	0.70	0.64	37
marakas	0.60	0.36	0.45	25
ocarina	0.48	0.37	0.42	27
piano	0.50	0.23	0.32	26
saxaphone	0.65	0.63	0.64	27
sitar	0.36	0.53	0.43	45
steel drum	0.36	0.29	0.32	28
trombone	0.53	0.54	0.54	35
trumpet	0.67	0.80	0.73	35
tuba	0.63	0.59	0.61	37
violin	0.57	0.26	0.36	31
accuracy			0.55	1019
macro avg	0.54	0.53	0.52	1019
weighted avg	0.55	0.55	0.54	1019

Accuracy Report

The following table summarizes the overall accuracy achieved by the five different models tested on the dataset:

Model	Accuracy (%)
XGBoost	55.05%
SVM (RBF Kernel)	56.62%
Extra Trees	53.09%
K-Nearest Neighbors (KNN)	48.77%
Random Forest	47.01%

Detailed Classification Metrics (Top Performer: SVM/XGBoost)

- **Bongo Drums:** Achieved high performance with a precision of 0.84 and recall of 0.90 in XGBoost.
- **Drums:** Consistently one of the best-performing classes with a recall of 0.97.
- **Difficult Classes:** Instruments like the Didgeridoo (recall 0.35) and Dulcimer (recall 0.11) proved much harder for the models to distinguish using HOG features alone.

Summary & Conclusion

The project successfully implemented a multi-class classification system for musical instruments. Among the models tested, Support Vector Machines (SVM) and XGBoost emerged as the strongest performers, both achieving over 55% accuracy.

Key Findings:

- **Feature Efficacy:** HOG features are effective for instruments with distinct geometric shapes (e.g., drums, bongo drums) but struggle with instruments that have complex or overlapping visual signatures.
- **Model Comparison:** Boosting (XGBoost) and Kernel-based methods (SVM) outperformed standard bagging techniques (Random Forest) and instance-based learning (KNN).
- **Class Imbalance/Similarity:** The confusion matrices indicate that certain wind instruments (like the flute and clarinet) are frequently misclassified as one another, suggesting that future work should explore deep learning (CNNs) to capture more intricate textures.

In conclusion, while classical ML with HOG provides a solid baseline for instrument classification, the ~56% accuracy ceiling suggests that the visual

diversity of the 30 classes requires more complex feature hierarchies.

Links:

Dataset: [30 Musical Instruments -Image Classification | Kaggle](#)